

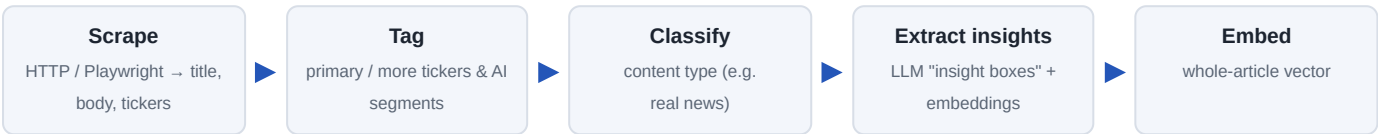
Ticker News Analyzer

A pipeline that turns financial news into searchable insights, then judges the immediate market reaction to a story — with a no-lookahead, backtest-honest design.

System overview · flows · retrieval selection · sentiment experiments

1 · The pipeline at a glance

Every article flows through five stages from raw URL to a fully enriched, embedded row. Each stage is **idempotent and resumable** (a run can be killed and restarted).



Source: Massive REST API (news list + minute/daily prices). LLM stages use Gemini; embeddings use OpenAI `text-embedding-3-small` (1536-dim).

2 · The database

Everything lives in one Postgres DB (`sharedproject` , on port `:15432`) with the **pgvector** extension. A content-poor local `news` DB exists as a fallback, so scripts always export `NEWS_DB_DSN` to point at the rich one.

Table	Holds
<code>articles</code>	one row per story: scrape fields, tickers, <code>category</code> , whole-article <code>embedding</code> , <code>has_insights</code>
<code>article_insights</code>	one row per extracted "insight box": topic, insight text, verbatim quotes, headline, per-box <code>embedding</code>
<code>ticker_data</code>	118-ticker universe → company name + AI segment
<code>ticker_overview</code>	Yahoo company descriptions (reference)

One shared vector space. Both `articles.embedding` and `article_insights.embedding` use the *same* model and 1536-dim space, so articles and insights are mutually searchable. Each gets an **HNSW cosine index** for fast nearest-neighbour search.

`has_insights` on `articles` is kept in sync by a trigger; an article counts as "insight-processed" once it has boxes *or* an `insights_extracted_at` stamp — so pure-boilerplate articles (which correctly yield zero boxes) are never re-sent to the LLM.

3 · Tagging, enrichment & classifying

Tag — tickers & segments (tag_segments.py)

Derives the `primary_ticker` and every *other* universe ticker mentioned — found via cashtags (`$NVDA`), exchange prefixes (`NASDAQ:` `NVDA`), parenthesised symbols, and company-name matching — plus each ticker's AI **segment**. Ambiguous symbols (`AI` , `ON` , `ARM`) only match in strong contexts to avoid false positives.

Classify — content type (classify_articles.py)

Labels each article into one `category` via a constrained Gemini enum, using a cost-aware **two-pass** strategy:

- **Pass 1** — fast/cheap `gemini-2.5-flash-lite` on every article.
- **Confirm** — only if pass 1 says `real news` , re-run on the stronger `gemini-2.5-flash` to keep or correct it. Other labels are accepted as-is.

The boundary that matters: **real news** = first-hand reporting of a *new* event; an explainer of an already-happened move ("why X fell today") is `recap/review` . Only `real news` feeds the sentiment experiments.

Enrich — insight boxes (extract_insights.py — the heart of the corpus)

Each article is asked for an array of **insight boxes** (a short topic, a 1–2 sentence takeaway with a bullish/bearish lean, and verbatim quotes). Key safeguards:

- **No hallucinated quotes** — every model quote is matched back to the source text (exact or fuzzy ≥ 0.75); non-matching quotes are dropped.
- **Context travels with the box** — the source headline is prepended and embedded, and tickers are cross-annotated (`NVDA` $\rightarrow$ `NVDA (NVIDIA)`).
- Each box becomes one embedded row in `article_insights` .

4 · Choosing the retrieval method

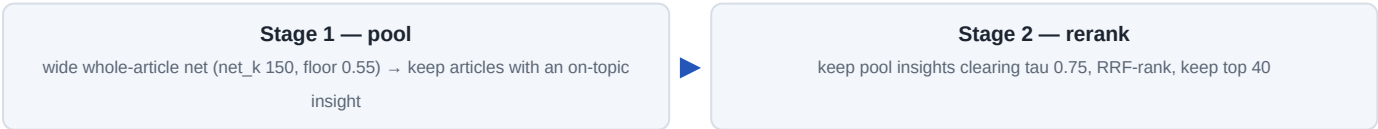
To judge a new article, the system retrieves *related prior coverage*. Two retrievers are complementary, and validation made the trade-off concrete:

Retriever	Searches	Strength
whole-article	<code>articles.embedding</code>	higher recall — nets the right documents
insight-level	<code>article_insights.embedding</code>	higher precision — matches the right <i>ideas</i>

The governing rule. Whichever method is applied *last as a hard gate* caps recall at that method's recall. So the high-recall method must build the wide net, and the high-precision method must filter inside it — never the reverse.

Selected method: two-phase-similarity

A two-stage hybrid that embodies the rule:



The **loose article floor (0.55)** fills the pool for recall; the **tight insight gate (0.75)** holds precision on what survives. Across both ground-truth tables and both evaluation modes it was the best all-rounder — strongest at the *insight* level (what the sentiment prompt actually consumes) and in the forward/cluster-recovery frame, degrading only to mid-pack in the strict no-lookahead corner. Default config: `net_min_sim 0.55` · `tau 0.75` · `budget 40` · `net_k 150` .

5 · Sentiment experiments — what worked, what didn't

The model reads the article + retrieved prior insights under a **strict timeline** (article = "now", prior insights = already-priced past) and returns **buy / sell / hold** + confidence. To isolate *article* signal from market drift, verdicts are tested on a "**peaceful-days**" pool (VIX < 25 and a flat 10-day tape) and scored against the realized buy → close return.

WORKED

A real, article-specific edge — and the two-phase retriever sharpens it

On 50 calm-day articles, BUY calls hit **~72%** (well above the 50% coin-flip) with the index move filtered out — so the verdict is reacting to the news, not market beta. `two-phase-similarity` was strictly better than the insight retriever: it kept all of its correct buys and added a few more (e.g. **AVGO +4.44%**, **LNVGY +2.43%**) that the simpler retriever had sat out as "hold".

DIDN'T

More "conviction" did not mean more accuracy

Two prompt levers were tested on 100–150 articles and **rejected by default**:

- `--include-strong` (adds `strong_buy/strong_sell`): made the model more decisive but *less* accurate — hit-rate fell **54%** → **47%**, and `strong_buy` was anti-calibrated (it *lost* money on average, worse than a plain buy).
- `--include-bias` (caveat that insights skew bullish): roughly neutral — it pruned some buys but the surviving buys had the *same* precision (~52%). A prompt caveat is a caution/volume knob, not an accuracy lever. Stacking both flags was the worst of all.

CONTEXT

Why "peaceful days" — the market-regime confound

The same model on a **bearish** window (Jan–Mar 2025, Nasdaq –10%) saw its BUY hit-rate collapse from ~72% toward coin-flip, while it began (correctly) issuing SELLS. Long-side accuracy is **regime-sensitive** — which is exactly why the calm-day filter exists: it holds the market direction flat so a hit-rate can be read as *signal* rather than drift.

Honest caveat. The realized metric is intraday (buy → publish-price → close) and the directional samples are small, so these read as *directional findings*, not significance tests. The durable takeaway is qualitative and consistent across runs: precision comes from the retrieval gates and the news itself — not from prompt embellishments.

Takeaways

- **The corpus is the moat.** Quote-verified insight boxes in a shared vector space make ideas — not just documents — retrievable, with no-lookahead windows for honest backtesting.
- **Retrieval design beats prompt tricks.** `two-phase-similarity` (loose recall pool → tight precision gate) is the selected method; prompt levers that add "conviction" did not improve correctness.
- **Measure against the regime.** A directional edge only means something once market drift is controlled for — the peaceful-days pool is the control that makes the ~72% BUY hit-rate meaningful.

Full detail: `docs/flows.md` · `docs/validations/validate_retrieval.md` · `docs/validations/sentiment_validation.md` . Models: Gemini 2.5 Flash / Flash-Lite (LLM stages), OpenAI text-embedding-3-small (vectors), Massive API (news & prices).